

Configurable Ensembles for Software Similarity: Challenging the Notion of Universal Metrics

1st anonymous author

Abstract—Software similarity analysis is crucial in various field, including code clone detection, security analyses, and software refactoring. While research continues to identify new use cases, numerous similarity detectors have already been proposed for specific contexts. These detectors usually leverage project attributes, such as source code, contributors, documentation, and dependencies. Existing works consistently demonstrate that their approaches outperform others in extensive evaluations. In this paper, we challenge the idea of a universally superior similarity model. We argue that similarity is a fluent concept and that relevant metrics always depend on specific needs. We present a novel framework that enables a flexible aggregation of diverse similarity models, allowing fine-tuned configurations for specific needs and use cases. Our evaluation incorporates multiple existing similarity models and their respective benchmarks to reveal the fundamental dilemma: depending on the configuration, our aggregated model will either confirm prior results or expose significant differences among individual models. However, we will demonstrate that these variations can be explained by the additional information that leads to more fine-grained results. Our results illustrate the future of software similarity research: configurable ensembles of much more specialized models.

Index Terms—Software similarity, multi-dimensional information, model aggregation, software project clustering

I. INTRODUCTION

Software similarity analysis is a fundamental area in software engineering (SE) with applications in code clone detection, vulnerability assessment, and code reuse [1]–[4]. Accurately identifying software similarities helps classify software projects, improve code reuse efficiency, and support large-scale software evolution studies [5], [6]. In large open-source platforms like GitHub, similarity analysis enables to track reuse patterns and detecting redundant codes, benefiting both individual developers and larger organizations [7], [8]. Many existing studies on software similarity have achieved significant progress, mostly focusing on specific aspects in software projects [2], [4], [9], such as source code, documentation, or dependency structures. Tools like CROSSIM [10] and MUDABLU [11] have been proven effective in identifying similarities between repositories. Normally, these studies aim to find an optimal approach for expressing software repository similarity and selecting the most suitable techniques. But calculating similarity between software systems is a complex task due to the diverse characteristics of software attributes. Different attributes influence similarity in distinct ways, making it challenging to develop a universal similarity model. For example, if two software projects written in different programming languages, the dependencies, coding styles, and

structural patterns may differ significantly, which complicates similarity calculations [12]. Therefore, when identifying similar software projects, the specific application objective can strongly influence the criteria of measuring similarity. For instance, in code plagiarism detection [13], the focus may be on code structure, syntactic patterns, or logic reuse. In contrast, for software recommendation or code reuse analysis [14], the emphasis might shift toward functional module similarity, common topics, or overlapping contributors. This diversity in requirements highlights the need for different similarity computation methods tailored to specific application contexts, rather than a one-size-fits-all approach.

We observed that most similarity tools focus on improving technical approaches to achieve the best similarity computation, or designing predefined methods to represent overall software similarity [10], [11], [15], [16]. These approaches often attempt to establish a universal similarity result applicable across different software projects. In these scenarios, software similarity is typically regarded as a fixed: a metric used to quantify the extent to which projects share common characteristics along predefined dimensions. However, we argue that similarity is not fixed or solely defined by tool-specific metrics. It depends on the task context, and varies based on the user's values. Thus, similarity should be understood as a context-aware, user-driven concept rather than a universal measure. As such, similarity measurement should be flexibly selected and adapted to match different scenarios and user needs. Although existing similarity models have demonstrated high effectiveness in their respective attributes, but lack a flexible and comprehensive framework to allows users to tailor the similarity computations to their needs, which limits the applicability of current tools.

To address this issue, we first analyze how existing similarity models operate and explore the feasibility, effectiveness, and specific methods for integrating different similarity models. Based on these insights, we propose a novel framework that enables flexible aggregation of multiple similarity models, allowing users to customize similarity computations based on different contexts. Furthermore, besides using previous similarity tools as a baseline model, we use build configuration files as an additional data source in the similarity aggregation. To achieve this, we answer the following research questions:

RQ₁ Do individual dimensions of repository data (e.g., code, documentation, dependencies, and activity data) share the same similarity characteristics?

We want to investigate to which extent existing similarity detection tools agree or how much similarity varies across different attributes, the results are discussed in Section IV.

RQ₂ When combining different dimensions of source data, does the resulting similarity clustering reveal more specific or targeted data?

We propose the Ensemble of Similarity Models (ESM), an approach to aggregate similarity values across multiple models. In Section III, we present the existing similarity model analysis, Section V discusses the core concept and methodology. Section VI evaluates its effectiveness by calculating the overlap ratio and provides a practical example.

The validation results indicate that the aggregated similarity results still maintain a high degree of overlap with single-attribute results. This confirms that the proposed approach still preserves the distinctive similarity characteristics of individual models while supporting multi-attribute integration. The main contributions of this paper are as follows:

- We highlight that integrating and assembling different similarity computation models should be the future direction.
- We explore build configuration as a new similarity attribute, which can also generalize to other similarity scenarios.
- We design a flexible similarity computation method that allows users to customize similarity measurements based on specific application needs. Experimental results demonstrate its effectiveness.

II. RELATED WORK

This section reviews current similarity research methods and examines the aggregation challenges across different models.

□ *Software Similarity.* Similarity analysis is an essential field in SE, developers benefit from learning similar style and functionality. It also plays a crucial role in various other domains, such as recommendation systems, plagiarism detection, and code clone detection [1]–[4]. However, a fundamental question in software similarity research remains: What defines “similar software”? The definition of software similarity is still debated. Some studies proposed that two applications are similar if they implement functionalities described by the same abstract concept [15]. Which follows Mizzaro's idea that documents sharing common concepts can be considered similar [17]. Many studies also directly use code similarity as the primary indicator of project similarity [5]. Especially in code clone detection, where similarity is often defined at the syntactic or structural level. For instance, existing study define clone as “a code segment that is identical to another segment” [18], and later work has expanded this to include segments that are syntactically or structurally similar [19]. Despite extensive discussion, there is no universally accepted definition of software similarity. As noted by Nguyen et al., different methods capture different aspects of similarity, making it difficult to establish a single definition for similarity in software [10].

□ *Artifacts Used in Similarity Research.* The definition of similarity is influenced by the chosen software artifacts, as each provides distinct measurement signals and affects applicability across different contexts. Source code is the most commonly used artifact, as it reflects a project's core implementation [20]. Other frequently analyzed artifacts include dependencies and textual documentation (README files/API docs), which offer descriptive or instructional context. Researchers have also explored management similarity [21], [22]. Some approaches utilize user activity such as stargazer behavior, shared contributors, commit patterns, and discussion frequency to capture inter-project relationships [20], [23], [24], providing complementary perspectives. In this study, we introduce build configuration files as a new dimension for similarity analysis. By capturing build tools, plugins, and dependency settings [25], they offer potential insights into how projects are structured and deployed. Prior research shows that different data sources often yield inconsistent similarity results. A key reason is that they convey different types of fundamental information. In particular, source code reflects structured implementation logic and is typically analyzed through syntax or control flow, whereas textual artifacts like README files convey semantic intent and project context. While both code and text can technically be stored as textual data, they serve different purposes. Code captures the structural and functional aspects, while text conveys semantic intent. As a result, text similarity may indicate application goals, whereas code similarity reflects implementation-level alignment, these two perspectives do not always align. For example, a Python and a Java web app may serve the same purpose but differ in structure; conversely, two Spring Boot apps may look structurally similar but target unrelated domains such as education vs. commercial use. Since different artifacts reveal distinct facets of a project, similarity should not rely on a single universal metric. Instead, it should be treated as context-dependent, shaped by user intent and the data source. A more effective approach is to build artifact-specific models and integrate them through a flexible framework that adapts to varied analysis needs, which aligns with the central goal of this study.

□ *Measurement of Similarity.* Once similarity is defined, the core challenge lies in measuring it effectively. Open-source projects involve diverse data types, requiring tailored detection techniques [16]. Prior studies typically focus on specific data sources or combinations, broadly can be categorized into code-based (structure/complexity) and semantic-based (README) approaches [20]. The choice of technique depends on the nature of the input data. Code-based methods may include machine learning (SVM/KNN) or semantic analysis (LDA/LSA). Methods based on project management data often use supervised learning or clustering techniques, such as weighted nearest neighbors or K-means [20]. Among numerous studies, is there an optimal method? While some tools achieve superior performance, the validation is often shaped by the tool's design goals and the input data type [10]. This raises our key motivation towards this research: How can we obtain

optimal similarity results when similarity goals do not align with existing models? To explore this, related research also provides insights. Case-Based Reasoning systems face similar challenges, where different similarity measures perform differently under varying conditions [26]. Studies also suggest that similarity measures can be optimized by assigning appropriate weights when they share the same scale [16]. Researchers have also combined syntactic and semantic measures to improve code clone detection [27]. A similar idea also applies to text and string analysis. Despite methodological differences, integrating multiple similarity measures consistently improves accuracy [28], [29]. Martinez's research further supports this, showing that different similarity measures complement each other, reducing limitations and enhancing performance [30]. We share this view that no single similarity model is universally optimal. Rather than focusing on depth within a single artifact, we focus on repository-level similarity across multiple heterogeneous artifacts, aiming to explore broader usage scenarios and flexible aggregation strategies.

□ *Challenges and Methods in Aggregating Similarities.* Comparing different models is inherently difficult due to differences in input types, structures, and computational logic. Model heterogeneity, inconsistent score scales, and varied evaluation criteria further complicate integration, especially when preserving each model's unique characteristics. As discussed, models vary in input types and computation logic. For instance, graph-based and deep learning approaches differ fundamentally in both representation and scoring, making their outputs difficult to compare or aggregate [31]–[33].

Additionally, the valuation standards are normally inconsistent, often relying on manual assessment or task-specific metrics, which limits comparability across studies [2], [9]. A model may perform well under its own criteria, yet its scores remain incompatible with others. While some work integrates multiple data sources using weighting or averaging [16], [23], these methods are often bound to specific design goals and lack generalizability. Therefore, the core challenge lies in how to effectively compare different similarity models within a unified framework while preserving their individual characteristics.

III. INSTANTIATING EXISTING MODELS

We instantiate and apply existing similarity models under consistent conditions to evaluate their agreement across different aspects (e.g., code, documentation, dependencies), addressing RQ₁. To broaden the analysis, we introduce build configuration files as an additional artifact. The following sections detail dataset selection, model implementation, and normalization. These steps also support RQ₂ by examining whether combining dimensions yields more refined or targeted clustering.

A. Dataset Collection

To ensure data reliability and reusability, this study uses the same dataset and criteria as the CROSSSIM experiments [33]: (i) GitHub Java projects; (ii) providing the specification of their dependencies by means of `pom.xml` or `.gradle` files; (iii) having at least 9 dependencies; (iv) having the `README.md`

file available; (v) possessing at least 20 stars. The original dataset contained 580 projects, some projects were no longer available, reducing the usable projects to 554. For build configurations analysis, we exclude GRADLE. Its flexibility and high degree of customization, leads to highly variable configurations. This lack of a consistent structure makes it difficult to extract uniform build steps across projects and introduces noise and inconsistency to the similarity analysis. To supplement build tool, we included two more Continuous Integration(CI) tools: TRAVIS CI and GITHUB ACTION. After filtering, we removed projects with empty configuration files, and 91 projects only use GRADLE and do not include any CI tools, the final dataset contains 446 projects, including 81 project use GitHub Actions, 116 projects use Travis CI.

B. Attributes Extraction

As discussed in the Section II, several artifacts are commonly used in similarity analysis. Based on this, we selected a set of prior models that are widely adopted and have publicly available implementations. These models cover multiple dimensions of software projects and reflect both structural and semantic perspectives.

As such, we selected MUDABLU to represent *Code*, collecting all source code files from the repositories. CROSSSIM was chosen for *Dependency* similarity, extracting information from `pom.xml` or `build.gradle` files. REPOPAL was used for *Documentation* similarity, retrieving all README files from the project root directories. Given that code, documentation, and dependencies are among the most commonly analyzed attributes in prior similarity research, we also incorporated *build configuration* as an additional factor for similarity analysis. and the following sections describe the process of extraction.

□ *Build configuration.* Since no existing similarity model directly supports build configurations, we extract build steps and custom commands from build files as the source data for similarity analysis. The configuration files include MAVEN (`effective-pom.xml`), GITHUB ACTION (`.github/workflows`), and TRAVIS CI (`.travis.yml`). The following sections outline the extraction process in detail. For details of the parser, refer to Github repository [34].

In MAVEN, for each project, we used MAVEN commands to generate the *effective pom* (to eliminates inheritance and properties) and the model API to parse this file [35]. MAVEN follows a fixed build phases, e.g., `compile` or `verify`, which users can customize by registering additional build plugins. We need to extract these plugins from the `<build>` section in the effective POM file and store their respective `groupId` and `artifactId`. We do not include plugin versions, as we do not expect that using different versions indicate a different intent of the developer. Similarly, we also do not consider `pluginManagement`, as it only affects the configuration, but not the activation of plugins. Developers can also define different `<profiles>`, which can activate in specific environments or conditions. We are conservative and exclude all build plugins that are referenced in any profile declaration.

In TRAVIS CI, developers define the project’s build process including configurations by using the `.travis.yml` file, normally located in the root directory of the repository. Travis CI also follows predefined stages, e.g., `before_install`, `install`, `before_script`, each stage consists of multiple parallel jobs [36]. Developers can customize build commands in different stages, and trigger by specific conditions, e.g., branches or tags. We use the SnakeYAML parser [37] to process the `.travis.yml` file and extract commands from each build stage under all trigger conditions, we collected all the commands as TRAVIS CI build configurations.

In GITHUB ACTION, Developers can automate build, test, and deployment workflows. The build process is defined in YAML files stored in the `.github/workflows/` directory. Each YAML file defines a separate workflow, specifying its execution logic through triggers, jobs, and steps. While workflows are typically independent, they can be interconnected through event-based triggers [38]. Each workflow consists of multiple jobs, where jobs can run independently, execute concurrently, or have explicit dependencies on one another. Steps in a job define individual execution units, which can include running shell commands, executing predefined GitHub Actions, or invoking custom scripts. To analyze GITHUB ACTIONS build processes, we developed a custom parser to extract from all YAML files in the `.github/workflows/` directory and standardize build steps. We extracted the concrete commands as GITHUB ACTION build configurations.

The extracted list of configuration step for each project is the input for the build file similarity analysis.

C. Similarity Matrix Creation

With all source data required by the four models extracted, this section now introduces the implementation of previous models, including the construction of similarity matrices for the novel build configuration similarity.

□ *Code*. MUDABLUe determines software similarity primarily through source code analysis. This method extracts identifiers from the source code and constructs an identifier-software matrix. It then applies Latent Semantic Analysis (LSA) and retrieves categories using similarities between identifiers. These identifier clusters are then mapped to software systems, forming software clusters that represent related software projects.

□ *Dependency*. CROSSSIM calculates repository similarity based on project relationships between dependencies and stargazer. It builds an OSS dependency graph, nodes are represented by projects, developers, and dependencies. The *SimRank* algorithm evaluates similarity by considering both direct and indirect relationships. The model incorporates shared dependencies, common contributors, and user interactions to measure repository similarity.

□ *Documentation*. REPOPAL calculated similarity by README files and user stargazer activities, it processes these sources separately, so to avoid overlap datasources with other models, we only use README files similarity calculator. It applies the Vector Space Model (VSM) to measure

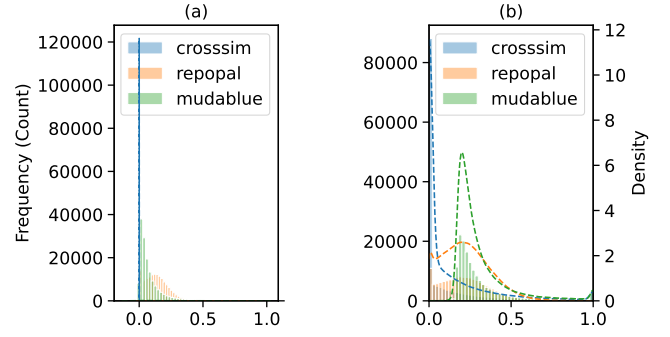


Fig. 1: Similarities, (a) before and (b) after normalization.

textual similarity between README files. Each README transforms into a TF-IDF weighted vector, and then calculated the cosine similarity score between repositories.

□ *Build Configurations*. After extracting configuration from the build configuration files, we standardize the data through the following cleaning procedures: (i) Remove URLs, user-defined folder paths, and custom variable names; (ii) Retain only the core components of frequently used commands, discarding specific options, e.g., in the command `javac -d path`, we omit the `-d` option and its associated argument, only keep the execution logic. Next, we merge build steps from different configuration files within the same project and remove duplicates. Then combine all extracted build steps into a single text document, representing the project’s overall build process. Then, we apply TF-IDF to convert them into numerical vectors. TF-IDF calculates the frequency of each word within a project while considering its distribution across all projects. This technique reduces the impact of frequently occurring but uninformative steps, and can emphasize the project-specific build characteristics. After vectorization, we compare the vectors of each project and use cosine similarity to generate the final similarity score.

For each attribute A , we compute a similarity matrix to represent pairwise repository similarities, serving as the basis for further analysis. The similarity matrix for attribute A is defined as following, where $S_{i,j}^A = \text{Sim}_A(\text{repo}_i, \text{repo}_j)$, represent $\text{repo } i$ and j similarity score in attribute A :

$$S^A = [S_{i,j}^A] \in \mathbb{R}^{n \times n},$$

D. Similarity Normalization

During the construction of the similarity matrix, we observed significant differences in the numerical distribution by similarity scores. For example, the dependency-based similarity model uses *SimRank* with a graph computation technique, its score range is very different from the textual similarity in RepoPal or the code analysis in MUDABLUe, as seen in Figure 1). The figure shows the frequency of similarity scores in each interval on the left y-axis (bars). The right y-axis shows the *Kernel Density Estimation* [39], which reflects the distribution of similarity values (dotted lines). CrossSim’s scores are

heavily concentrated near zero due to fundamental differences in its computation method, making direct comparison with the other tools difficult before normalization. To ensure that similarity scores from different methods maintain a consistent scale and to mitigate the impact of extreme values, we apply several normalization steps to the similarity matrix S^A to ensure a uniform scale for further analyses:

□ *Numerical Preprocessing.* We remove extreme values similarity matrix S^A , using interquartile ranges [40]. Any values that fall outside the calculated lower and upper bounds are clipped to these respective limits, which reduces the influence of outliers on similarity computation. In contrast to the conventional approach, we do not select the first and third quartile, but utilize the 0.1 and 0.9 quantile. This modification is more suitable for datasets, as it retains more information while still mitigating the impact of extreme outliers.

We observed a strong prevalence of zero values in some similarities. We preserve zeroes in the following two cases: *First*, a zero value indicates no similarity between objects. This assumption holds across different methods. For example, in dependency graph calculations, if two nodes share no common connections, the computed similarity score is zero, reflecting low similarity. *Second*, zero value may result from insufficient data to support similarity computation. For example, a missing README file suggests that its documentation similarity with properly documented projects is low or even zero.

□ *Z-Score Normalization.* After the preprocessing, we apply Z-Score normalization to ensure that all similarity scores have a mean of 0 and a standard deviation of 1. While most similarity models produce values within the $[0, 1]$ range, a different numerical scale can be misleading in direct comparisons.

$$z = \frac{S^A - \text{mean}(S^A)}{\text{std}(S^A) + 1e^{-6}}$$

Here, $\text{mean}(S^A)$ represents the mean similarity score within matrix S^A , and $\text{std}(S^A)$ denotes the standard deviation. Normalization ensures that all similarity scores are centered around 0, preventing certain methods from disproportionately influencing the analysis due to scale differences.

□ *Min-Max Normalization.* To further ensure that all similarity computation methods produce outputs on the same scale, we apply Min-Max normalization [41], scaling all similarity values into the $[0, 1]$ range.

$$S^{A'} = \frac{S^A - \min(S^A)}{\max(S^A) - \min(S^A) + 1e^{-6}}$$

Here, $\min(S^A)$ and $\max(S^A)$ represent the minimum and maximum values in matrix S^A , respectively, and $S^{A'}$ is the normalized similarity matrix. Min-Max normalization ensures that all similarity scores fall within the $[0, 1]$ range, eliminating variations caused by different numerical scales. These steps allows results from different methods to be directly compared and used for clustering analysis. The distribution of normalized similarity values is shown in Figure 1(b). After normalization, all similarity scores are scaled to the unified $[0, 1]$ range. As shown in Figure 1(b), CrossSim's

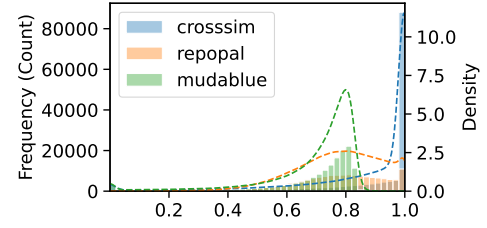


Fig. 2: Normalized Distance (frequency: bar, density: line))

values are redistributed into a more comparable range, and the distributions patterns of RepoPal and MUDABlue become more amplified. The normalization process ensures similarity scores from all computation methods are within a consistent numerical range, enabling direct comparisons across methods. The final normalized similarity matrix serves as the input for subsequent modeling analysis.

IV. SIMILARITY OF EXISTING MODELS (RQ1)

Although different similarity models achieve high effectiveness in their respective objectives, their behavior across different attributes may vary significantly. Investigating these differences and their underlying factors is a core motivation of this study: We argue that an optimal representation of similarity should not be unique.

□ *Methodology.* In Experiment Section III, we extracted the required data from previous models and re-ran similarity models based on documentation, code, and dependencies. We created an additional model to measure build similarity. After generating the corresponding similarity matrices, we applied normalization to produce the final similarity matrix. Finally, we compare normalized similarity results across three aspects: ranking consistency, numerical differences, and structural alignment. These aspects collectively illustrate the differences between models. The following sections discuss the experimental results in detail.

□ *Result.* Following list the detailed differences:

Ranking Similarity. To assess the ranking consistency between the similarity matrices, we employed Spearman's Rank Correlation Coefficient [42]. We treat similarity scores as continuous variables and determine their ranks based on natural ordering within the full set. Which leverages the relative order of all item pairs, capturing subtle differences in global ranking between models. This non-parametric measure evaluates the statistical dependence between the rankings of two variables, assessing how well the relationship between two variables can be described by a monotonic function. Spearman's correlation coefficient ranges from +1 (perfect positive correlation) to 0 (no correlation) and -1 (perfect negative correlation). By focusing on the rankings rather than the raw values, this method minimizes the impact of zeros in the matrices, ensuring that the presence or absence of specific similarities does not overly influence the final result. After performing 3000 iterations of permutation testing to reduce effect of sparse values or zero values in the similarity matrices, we observed all p-values were low (< 0.05) confirming that the

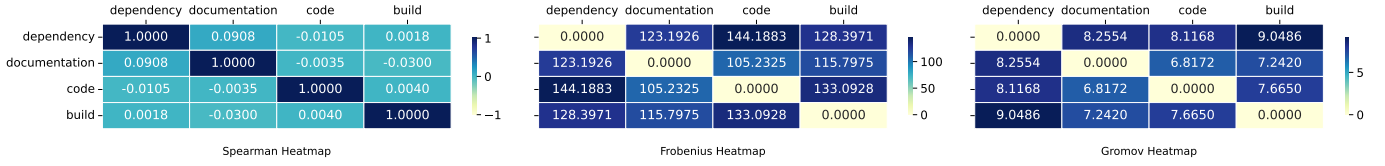


Fig. 3: Similarity matrix comparison

results were not due to random fluctuations. But the observed Spearman correlation coefficients indicate weak relationships. The correlation between CROSSSIM and REPOPAL is 0.09, suggesting a slight positive correlation in ranking, CROSSSIM and MUDABLU exhibit a weak negative correlation (-0.01), while REPOPAL and MUDABLU also show very low relate. The coefficients between BUILD and other models are also low, indicating the similarity rankings are nearly uncorrelated between the models. A weak correlation does not imply the models are ineffective. Instead, it indicates that they are largely independent and capture different aspects of similarity. All models strive to identify the most similar projects within their respective contexts, their rankings can vary differently.

Numerical Similarity. To evaluate the numerical differences between the matrices, we used the Frobenius Norm [43], which calculates the square root of the sum of squared differences between corresponding elements in the matrices. This provides a measure of the overall numerical distance between two matrices. The results demonstrated significant numerical differences across the three methods, all three matrices were normalized before comparison, but the result also shows the numerical difference. Specifically, CROSSSIM exhibits the largest numerical disparity when compared to both REPOPAL and MUDABLU. MUDABLU and REPOPAL demonstrate the smallest numerical difference relatively. For BUILD, we observe relatively large Frobenius distances from CROSSSIM (128.4) and MUDABLU (133.1), suggesting differences in the way similarity is constructed. The distance to REPOPAL is slightly smaller (115.8).

Structural Similarity. To assess the geometric structure of similarity matrices, we use the Gromov-Hausdorff distance [44], which measures the minimum distortion needed to embed two metric spaces into a common space. It is especially useful when these spaces vary in size, dimensions, or topology. In our study, the diverse input features lead to variations in sparsity, or clustering patterns across similarity matrices. The Gromov-Hausdorff distance captures these structural deviations within a common embedding space, allowing us to reveal fundamental differences in the underlying “similarity structure” of each model, which can reduce the input distribution and support more balanced comparison.

The results indicate CROSSSIM exhibits largest structural differences from both MUDABLU and REPOPAL. REPOPAL and MUDABLU share the closest structural similarity, as their geometric distance is the smallest. This trend is consistent with the Frobenius Norm patterns findings. Besides, BUILD exhibits relatively small Gromov-Hausdorff distances with REPOPAL (7.2) and MUDABLU (7.7), indicating a certain degree of structural proximity, in contrast, although CROSSSIM operates

on dependency data, the differences in algorithm design and implementation contribute to its divergence from other models.

The observed variations across similarity measures spanning code, documentation, dependencies, and build configurations, highlight their distinct design orientations. CROSSSIM diverges significantly from other models, likely due to its graph-based method rather than semantics. The consistently low Spearman correlations reinforce the idea that these models capture complementary dimensions of repository similarity, rather than measuring the same construct. Although build configuration shows limited correlation, it reveals moderate numerical and structural differences in Frobenius and Gromov-Hausdorff analyses, underscoring its value as a distinct perspective. These divergences motivate ensemble modeling to harness the complementarity of heterogeneous approaches.

V. USING AN ENSEMBLE OF SIMILARITY MODELS

In the conclusions drawn from RQ1, we validated our initial hypothesis that different similarity models perform variably across data attributes. This highlights a fundamental limitation of relying on a single or predefined similarity model, which inherently follows its original design objectives, it may fail to comprehensively capture the diverse similarity computation needs across different scenarios. While certain similarity models may perform well for specific attributes, their effectiveness is often constrained when applied to other attributes.

This underscores the necessity of a adaptable similarity computation framework, capable of leveraging multiple models to account for different aspects of similarity. To address this limitation, we propose an ensemble similarity modeling approach, referred to as the Ensemble of Similarity Models (ESM). This method employs a multi-dimensional attribute aggregation strategy, integrating multiple similarity models to construct a more interpretable and flexible similarity computation framework. The following sections will outline the key challenges encountered and the core concept and solutions devised to address to them accordingly.

□ *Key Challenges and Solutions.* As discussed, the primary challenge in similarity computation is the disparity in the methodologies and output formats of different similarity models, making direct comparison and aggregation infeasible. Since different similarity models adopt diverse computational techniques, a fundamental requirement is that the similarity model at least need to generate a similarity result, explicitly representing similarity scores between repositories. This requirement ensures that, regardless of internal model operations, their outputs can be structurally aligned, thereby enabling cross-model compatibility and fusion.

To prevent information loss from simple averaging or direct score aggregation, we map similarity values from different attributes into a multi-dimensional space. Each similarity attribute is treated as an independent dimension, allowing the system to preserve the distinct characteristics of different models while conducting analysis in a unified computational space. This ensures that the contributions of individual models remain distinguishable, enhancing the interpretability and granularity of the similarity computation. In the aggregated multi-dimensional space, we compute similarity scores using Euclidean distance [45], [46]. This metric inherently measures the straight-line distance between points in a multi-dimensional space, it naturally accounts for differences in similarity scales, while ensuring that no single similarity model disproportionately influences the final computation, this allows an effective and geometrically consistent similarity computation across multiple models, acts as a standardization mechanism, ensuring a balanced and scalable aggregation.

□ *Methodological Steps.* The process starts with multiple similarity models computing similarity scores for the target dataset. Each model produces a similarity matrix S^A for attribute A , where $s_{i,j}^A$ denotes the similarity score between repositories i and j . All matrices are of size $N \times N$, where N is the number of repositories. All matrices are pre-normalized to ensure consistent scaling across models, as detailed in Section III. Normalization maps raw scores to the $[0,1]$ range, since Euclidean distance is sensitive to scale differences, this prevents any single model from dominating the result and ensures that $d_{i,j}$ provides a fair and geometrically consistent measure of cross-model similarity. For the similarity matrices $S^A, S^B, \dots, S^N$ generated by different models for various attributes, we aggregated them into a unified multi-dimensional representation. For each repository pair (i, j) , we define a multi-dimensional similarity vector.

$$\vec{s}_{i,j}^{A,B} = (s_{i,j}^A, s_{i,j}^B, \dots, s_{i,j}^N)$$

where each component $s_{i,j}^k$ corresponds to the similarity score obtained from the k -th similarity model. This transformation ensures that different similarity characteristics still preserves its individual contributions.

To compute the final similarity, we transform the similarity vector representation into a distance representation and measure the global similarity between repository pairs using Euclidean distance in the multi-dimensional similarity space:

$$d_{i,j} = \sqrt{\sum_{k=1}^M (1 - \vec{s}_{i,j}^{A,B})^2}$$

where $d_{i,j}$ represents the final distance for repository pair (i, j) . The aggregated similarity score is $s_{i,j} = 1 - d_{i,j}$.

VI. EVALUATION OF ESM (RQ2)

This study introduces a method for combining different computational models. A key challenge is ensuring that the merged results retain the characteristics of the original similarity models. We aim to investigate whether aggregating similarity across different source artifacts leads to more refined results

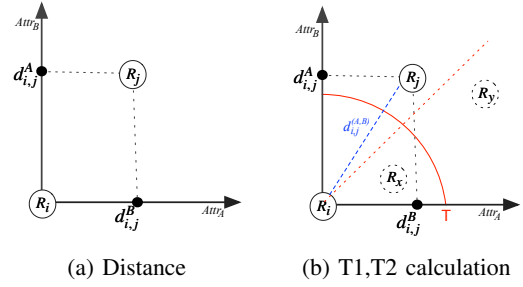


Fig. 4: Distance measure

and to analyze the specific changes, validating the effectiveness of this approach is a key motivation of this study.

A. Evaluation Methodology

We primarily use clustering to validate our approach. We analyze the overlap ratio between clustering results before and after attribute aggregation. To achieve this, we employ the Canopy clustering algorithm [47], which is recognized for its simplicity and efficiency, and rely on distance as the core metric for clustering. Specifically, our validation consists of the following steps: First, we aggregated similarity from different similarity models, then we perform clustering on the similarity matrices for both single-attribute and multi-attribute data. Next, we compute the o score to assess the structural similarity between the clustering results of aggregated attributes and those of individual attributes. Our hypothesis is that after merging multiple attributes, the clustering structure should become more fine-grained. In the following sections, we provide a detailed discussion on the clustering process and present the methodology for computing clustering results and measuring the overlap score between model aggregations.

□ *Canopy Clustering.* We perform Canopy clustering to cluster repositories according to their attributes. Canopy clustering has two parameters T_1 and T_2 that can be used to control the clustering results. The algorithm picks a random point as the first cluster center and finds all other points, for which the distance $d \leq T_1$. These points will be assigned to the current cluster. In case distance d is even smaller than T_2 , the point is eliminated as a candidate for new Canopy centers. The algorithm terminates when no further candidates exist.

Normally, Canopy clustering is a *soft clustering*, i.e., one data point can be assigned to multiple clusters. We set $T = T_1 = T_2$ to achieve a *hard clustering*, so each data point is assigned to exactly one cluster. This ensures non-overlapping clusters, reduces ambiguity in cluster membership, and creates distinct boundaries between clusters.

We automatically select an appropriate T value from a dynamically generated range: We compute all pairwise distances from the upper triangle (excluding the diagonal) of the distance matrix and flatten them into a 1D array. We select the mean plus/minus three times the standard deviation [40], [48] of these distances as min/max for T , which can capture most of the distance values, and focus on the central range of the data distribution and avoid extremely low/high values.

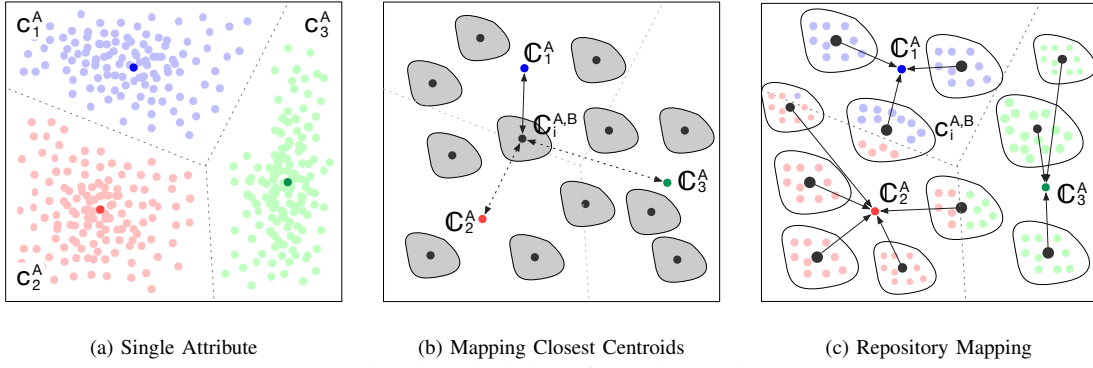


Fig. 5: Calculation of Overlap Ratio

Within the defined range, we generate 20 equally spaced T values using linear interpolation to ensure a good coverage of various thresholds. To find the optimal T value, we then use the Silhouette Coefficient s as the evaluation metric, which is a widely used metric for assessing clustering quality [49]. It is calculated as: $\xi = (b - a) / \max(a, b)$, with a being the average distance between a data point and all other points in the same cluster (intra-cluster distance) and b the average distance between the same point and all points in the nearest neighboring cluster (nearest-cluster distance). This coefficient measures clustering consistency in a range from -1 to 1, higher values indicate more cohesive clusters and better performance.

□ *Overlap Ratio*. To evaluate whether the clustering structure of a single attribute is preserved after merging multiple attributes, we introduce the metric *Overlap Ratio*. It measures how well multi-attribute clustering retains the original single-attribute structure while enabling more fine-grained classification. The core idea is that combining multiple attributes should refine the clustering without losing the essential characteristics of the original clusters. To explain the calculation, we first need to introduce certain terms. Considering a clustering result C^A based on the single attribute A and producing three clusters c_{1-3}^A . Each of these cluster c_i^A contains one or more repositories r . We cannot calculate the geometric center for a cluster, as some models do not allow a spatial interpretation, as such, we determine the *approximate centroid* $\mathbb{C}_n^A$ by finding the repository with the minimal total distance to all other repositories in the cluster. The actual calculation of the overlap ratio is illustrated in Figure 5. We see the three hypothetical clusters c_i^A in Figure 5a. Each dot in the cluster represents one repository, the darker dots represent the centroids. The goal is to compare the stability of this *base* clustering to an *extended* clustering in a higher dimension. The example contains a clustering along the attributes A, B , which results in more and smaller clusters, as depicted by the grey areas in Figure 5b. Each of these new clusters has a respective centroid again, for example, $\mathbb{C}_i^{A,B}$ in the center of the figure. The goal of the *Overlap Ratio* is now to quantify how the extended clustering compares to the base clustering of the previous step.

The algorithm requires two utility functions. $f_A^{A,B}$ maps a centroid (i.e., a representative repository) of an extended cluster to the closest centroid of any of the base clusters.

The look-up function g can select the whole cluster (i.e., all contained repositories) given its representative centroid.

$$f_A^{A,B} : \mathbb{C}^{A,B} \rightarrow \mathbb{C}^A$$

$$g : \mathbb{C}_i \rightarrow c_i$$

By concatenating both functions, $\hat{c}$ returns a corresponding base cluster for a given extended centroid.

$$\hat{c}_i^{A,B}(i) = g \circ f_A^{A,B}(\mathbb{C}_i^{A,B})$$

In Figure 5b, $\mathbb{C}_1^{A,B}$ is the closest centroid for $\mathbb{C}_i^{A,B}$ and $\hat{c}$ would contain all repositories of cluster c_1^A .

The overlap ratio $\mathbb{O}_A^{A,B}$ of *one cluster* with respect to the dimensionality reduction from A, B to A can now be calculated by dividing the number of repositories in the extended cluster that also exist in the matching base cluster by the total number of repositories in the extended cluster. The blue repositories in Figure 5c have been assigned to cluster c_1^A before, while the red repositories were part of c_2^A before.

$$\mathbb{O}_A^{A,B}(i) = \frac{|c_i^{A,B} \cap \hat{c}_i^{A,B}(i)|}{|c_i^{A,B}|}$$

The overlap ratio $\mathbb{O}_A^{A,B}$ that represents the average overlap of all clusters with respect to the dimensionality reduction from A, B to A is then calculated as:

$$\mathbb{O}_A^{A,B} = \frac{1}{n} \sum_{i=0}^n \mathbb{O}_A^{A,B}(i)$$

For simplicity, these examples were only two-dimensional and used a single reduction (i.e., A, B to A). However, the concept holds for n -dimensions. To measure the overlap ratio for a cluster $c_i^{A,B,C}$, it is necessary to compute all the possible reductions $\mathbb{O}_{A,B}^{A,B,C}$, $\mathbb{O}_{B,C}^{A,B,C}$, and $\mathbb{O}_{A,C}^{A,B,C}$. The global overlap ratio $\overline{\mathbb{O}}$ is then average overlap of all possible reductions.

$$\overline{\mathbb{O}}^{A,B,\dots,n} = \mathbb{O}(\mathbb{O}_{B,\dots,n}^{A,B,\dots,n}, \mathbb{O}_{A,C,\dots,n}^{A,B,\dots,n}, \dots, \mathbb{O}_{A,B,\dots,n-1}^{A,B,\dots,n})$$

□ *Overlap Result*. Table I and Table II present the computed overlap ratio across different dimensional comparisons,

Table I illustrates the overlap ratio between one-dimensional base attributes and the corresponding two-dimensional extensions. The base attributes include C (Code), B (Build), D (Dependency), and T (Text/Documentation). Each cell represents the overlap ratio of aggregation attributes compared to base attribute. For instance, For instance, the value in the first row, second column represents $\mathbb{O}_C^{C,D}$, which the overlap ratio

between the clustering structure of the (Code,Dependency) aggregation and the original Code attribute.

Table II expands this analysis to three-dimensional clustering configurations, comparing the structural consistency between two-dimensional base clustering and three-dimensional clustering. Each row in the extended clustering column represents a clustering structure formed by introducing an additional attribute to an existing base clustering. For example, the first row represent the overlap ratio $\mathbb{O}_{B,C}^{C,B,D}$, which represents a two-dimensional clustering of Code and Documentation, and its corresponding three-dimensional variant is obtained by further incorporating Dependency.

From a general perspective, the silhouette coefficient all falls within the average range of 0.3 to 0.6. Although there is no absolute threshold, a value of 0.5 or above is generally considered indicative of well-separated and well-formed clusters. Still, clustering validity also depends on the specific characteristics of the dataset and the clustering algorithm used. Additionally, during the experiments, we found that higher silhouette scores often correlated with better clustering and higher overlap ratios. In this study, we employ Canopy clustering, due to its inherent nature, Canopy clustering does not necessarily maximize inter-cluster separability, which can lead to relatively moderate silhouette scores. Base on this, the observed range of silhouette coefficients suggests that the clustering structure is still meaningful. Given the exploratory nature of this study, these results remain valid and provide a reliable foundation for further comparative analysis across different similarity models.

Overlap ratio. Table I and Table II show a consistent high overlap ratio, with an average exceeding 80%. This evaluation in Table I includes comparisons across all individual attributes to aggregated attributes involved in the aggregation process. For instance, in the combination of documentation and code, we compare the aggregated attributes separately with each of the individual dimensions (i.e., documentation and code), and both exhibit a high overlap ratio. Similarly, in the comparison between two-dimensional and three-dimensional attribute aggregations, we observe the same trend. The consistently high overlap ratio across different attribute combinations suggests that after mapping all attributes into a unified space, the clustering results of individual attributes become more finely segmented upon aggregation. From the definition of the overlap ratio, this indicates that the structural integrity of single-attribute clustering is preserved while achieving finer-grained distinctions in the aggregated space.

We observe that multi-attribute clustering refines project groupings, leading to more structured and granular clusters, this finding validates our initial hypothesis and the proposed approach. After multi-dimensional aggregation, the clustering becomes more refined while still preserving the structure of individual attributes. In response to RQ2, we confirm that attribute aggregation is effective in combining similarity models to apply to different needs.

TABLE I: One vs Two dimension Overlap Ratio (%)

Base Attribute	Added Attribute			
	Code	Dependency	Documentation	Build
Code	100.0	82.7	85.1	83.6
Dependency	87.4	100.0	89.2	88.4
Documentation	87.1	85.3	100.0	84.7
Build	83.3	85.9	88.1	100

TABLE II: Two vs Three dimension Overlap Ratio(%)

Extended Clustering	Base Clustering	$\mathbb{O}$
Code, Build, Dependency	Code, Build	82.0
	Code, Dependency	85.5
	Build, Dependency	87.8
Build, Dependency, Documentation	Build, Dependency	90.2
	Documentation, Build	84.1
	Documentation, Dependency	86.7
Code, Build, Documentation	Code, Build	84.5
	Code, Documentation	85.3
	Build, Documentation	85.1
Code, Dependency, Documentation	Code, Dependency	86.2
	Code, Documentation	82.2
	Dependency, Documentation	86.5

□ *Case Study.* To evaluate the practical value of ESM, we conducted a preliminary case study on 97 manually labeled projects belonging to four families: 17 commercial Spring applications, 47 Spring Boot plugins, 24 Minecraft plugins, and 9 personal projects used for demos or tutorials. These projects differ in their typical usage scenarios. To investigate each model’s ability to capture intra-family consistency among projects, we computed similarity scores for each repository and examined whether its top-ranked results belonged to the same family. Specifically, for each project $repo_i$, we used each model’s similarity matrix to identify its top five most similar projects and calculated the proportion that shared the same family label. We then averaged these proportions across all projects to derive an overall score that reflects each model’s ability to preserve family-level cohesion. In this evaluation, ESM achieved a score of 83.9%, outperforming `repopal` (72.0%), `crosssim` (69.1%), `mudablue` (32.8%), and `build` (59.0%). (Detailed results seen in Github [34]). These results suggest that projects within the same family share consistent similarity patterns, and ESM is better at capturing them than single-metric models.

VII. DISCUSSION

This section reflects on key findings, discusses the implications, and outlines limitations and directions for future work.

□ *Ensembles Enable Flexible Similarity Evaluation.* Our study shows that predefined single-model similarity approaches perform well in specific scenarios, but lack adaptability. As seen in Section IV, model differences in algorithms, input types, and scoring scales hinder direct comparison. Future research

should prioritize precise calculations for specific attributes, instead of searching for a one-size-fits-all similarity computation method, or seeking a single universal metric to represent similarity. Ensemble methods allow multiple specialized models to be combined dynamically, creating a more flexible and context-aware similarity evaluation framework. This approach represents the direction of similarity computation, focusing on user-driven integration.

□ *Overlap Ratio.* We introduce overlap ratio to address a key validation challenge. It quantifies the consistency between clustering structures in aggregated similarity results and individual similarity models. This metric provides an objective method to determine whether multi-attribute similarity preserves the structural integrity of single-attribute measurements. This measurement can also be extended to supplement other validation tasks in multi-attribute similarity analysis.

□ *Expanding the Dimensions of Similarity.* In this study, we explored a basic method to compute similarity from build configuration files by treating them as text and applying TF-IDF to extract feature vectors. This enables us to construct similarity based on the extracted textual patterns of build configurations. While still preliminary, this research lays the foundation for future improvements. It introduces a new perspective on similarity computation, suggesting that build configuration files can serve as an independent similarity metric. Integrating this into broader frameworks may enhance insights into software evolution and development practices. Future work should aim to improve the precision of this method.

□ *Weighting in Similarity Ensemble.* Another crucial aspect is the development of adaptive weighting strategies in similarity computations. Dynamic weights tailored to specific application needs, can significantly improve the flexibility and effectiveness of similarity models. To improve the reliability, researchers also need to develop robust validation techniques that accurately reflect the relative importance of different attributes. Combining attribute-specific models with adaptive weighting enables more precise and context-aware evaluation, enhancing applicability across diverse software scenarios.

□ *User-Guided Similarity Adaptation.* While this work establishes a multi-dimensional framework, for more practical usage, the effectiveness should inherently depend on its end-use context, which acts as a bridge between user intent and technical capability. In real-world applications, the semantics of “similarity” are not fixed but depend on the task at hand. For example, in recommendation scenarios, the similarity model should align closely with the user’s intent and allow flexible composition, thereby enabling user-centric decision-making. For tasks such as code reuse detection, the emphasis may shift toward structural equivalence or alignment at the dependency level. Importantly, the concept of “user” here extends beyond human developers or end-users. It also includes system components or agents that perform filtering or recommendation roles in specific tasks. Therefore, designing similarity mechanisms centered on “user roles” can help to

improve context-awareness and task alignment in downstream applications. Large Language Models (LLMs) further provide possibilities in this direction. With their strong capabilities in processing unstructured semantic content, LLMs are well-suited to serve as high-level similarity models. They can potentially capture latent semantic associations and trace evolutionary relationships across software projects, making them a promising avenue for future exploration.

□ *Generalizability in Similarity Computation.* By analyzing similarity across different attributes, we can create more diverse datasets to replicate previous studies and test their generalizability. Reproducing prior research on datasets of varying types helps evaluate the robustness of existing methods in broader environments. This ensures that similarity models do not overly rely on specific datasets, improving their adaptability across different software ecosystems.

□ *Threats to Validity.* For internal validity, we used Canopy clustering due to efficiency. It requires only distance thresholds and works well with stable scores, making it suitable for our similarity input. However, as a basic method, it may not capture more complex clustering structures. Future work may explore advanced clustering algorithms to enhance accuracy and validation rigor. Our method aggregates similarity scores using Euclidean distance without further transformation. While Euclidean distance is widely accepted for multi-dimensional integration, exploring alternative distance metrics may improve future similarity computations. For external validity, our approach depends on the quality of input similarity scores, making model selection important. Only models that produce interpretable similarity scores can be integrated, which may exclude some systems. Although we selected widely-used, validated models, biases or inaccuracies in any input model can affect the final output. Moreover, our evaluation is based on open-source Java projects. Similarity patterns may differ in other ecosystems, languages, or development communities. To improve generalizability, future studies should evaluate the framework across diverse datasets and project types.

VIII. SUMMARY

Software similarity analysis plays a key role in software engineering. Existing approaches often rely on predefined attributes, limiting their adaptability to diverse application needs. This study presents a flexible framework that integrates multiple similarity models, enabling context-aware and customizable similarity computation. Our evaluation confirms that the aggregated results preserve the structural patterns of individual models while offering broader applicability. These findings emphasize the importance of adaptive similarity frameworks that align with varying user perspectives and tasks. Our implementation is publicly available on GitHub [34].

REFERENCES

- [1] A. Walenstein and A. Lakhotia, “The software similarity problem in malware analysis.” Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2007.

- [2] K. W. Nafi, B. Roy, C. K. Roy, and K. A. Schneider, "A universal cross language software similarity detector for open source software categorization," *Journal of Systems and Software*, vol. 162, 2020.
- [3] C. Liu, C. Chen, J. Han, and P. S. Yu, "Gplag: detection of software plagiarism by program dependence graph analysis," in *ACM SIGKDD international conference on Knowledge discovery and data mining*, 2006, pp. 872–881.
- [4] A. Michail and D. Notkin, "Assessing software libraries by browsing similar classes and relationships," in *Proceedings of the 21st international conference on Software engineering*, 1999, pp. 463–472.
- [5] P. T. Nguyen, J. Di Rocco, and D. Di Ruscio, "Mining software repositories to support oss developers: A recommender systems approach," in *IIR*, 2018.
- [6] M. Gharehyazie, B. Ray, and V. Filkov, "Some from here, some from there: Cross-project code reuse in github," in *International Conference on Mining Software Repositories (MSR)*. IEEE, 2017, pp. 291–301.
- [7] A. Mockus, "Large-scale code reuse in open source software," in *First International Workshop on Emerging Trends in FLOSS Research and Development (FLOSS'07: ICSE Workshops 2007)*. IEEE, 2007.
- [8] L. Ponzanelli, G. Bavota, M. Di Penta, R. Oliveto, and M. Lanza, "Mining stackoverflow to turn the ide into a self-confident programming prompter," in *Proceedings of the 11th working conference on mining software repositories*, 2014, pp. 102–111.
- [9] A. Capiluppi, D. Di Ruscio, J. Di Rocco, P. T. Nguyen, and N. Aijenka, "Detecting java software similarities by using different clustering techniques," *Information and Software Technology*, vol. 122, 2020.
- [10] P. T. Nguyen, J. Di Rocco, R. Rubel, and D. Di Ruscio, "An automated approach to assess the similarity of github repositories," *Software Quality Journal*, vol. 28, pp. 595–631, 2020.
- [11] S. Kawaguchi, P. K. Garg, M. Matsushita, and K. Inoue, "Mudablue: An automatic categorization system for open source repositories," in *11th Asia-Pacific Software Engineering Conference*. IEEE, 2004.
- [12] K. W. Nafi, B. Roy, C. K. Roy, and K. A. Schneider, "Crolsim: Cross language software similarity detector using api documentation," in *2018 IEEE 18th international working conference on source code analysis and manipulation (SCAM)*. IEEE, 2018, pp. 139–148.
- [13] G. Cosma and M. Joy, "An approach to source-code plagiarism detection and investigation using latent semantic analysis," *IEEE transactions on computers*, vol. 61, no. 3, pp. 379–394, 2011.
- [14] A. Ouni, R. G. Kula, M. Kessentini, T. Ishio, D. M. German, and K. Inoue, "Search-based software library recommendation using multi-objective optimization," *Information and Software Technology*, 2017.
- [15] C. McMillan, M. Grechanik, and D. Poshyanyk, "Detecting similar software applications," in *International Conference on Software Engineering (ICSE)*. IEEE, 2012, pp. 364–374.
- [16] M. O. F. Rokon, P. Yan, R. Islam, and M. Faloutsos, "Repo2vec: A comprehensive embedding approach for determining repository similarity," in *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2021, pp. 355–365.
- [17] S. Mizzaro, "How many relevances in information retrieval?" *Interacting with computers*, vol. 10, no. 3, pp. 303–320, 1998.
- [18] I. D. Baxter, A. Yahin, L. Moura, M. Sant'Anna, and L. Bier, "Clone detection using abstract syntax trees," in *Proceedings. International Conference on Software Maintenance*. IEEE, 1998, pp. 368–377.
- [19] Q. U. Ain, W. H. Butt, M. W. Anwar, F. Azam, and B. Maqbool, "A systematic review on code clone detection," *IEEE access*, 2019.
- [20] "Similarity-based analyses on software applications: A systematic literature review," *Journal of Systems and Software*, vol. 168, 2020.
- [21] C. T. Creel and O. A. Chappel, "Project management system and method for assessing relationships between current and historical projects," Apr. 29 2008, uS Patent 7,366,680.
- [22] M. Ellmann, "On the similarity of software development documentation," in *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, 2017, pp. 1030–1033.
- [23] Y. Zhang, D. Lo, P. S. Kochhar, X. Xia, Q. Li, and J. Sun, "Detecting similar repositories on github," in *2017 IEEE 24th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2017, pp. 13–23.
- [24] I. Guy, N. Zwerdling, D. Carmel, I. Ronen, E. Uziel, S. Yogeve, and S. Ofek-Koifman, "Personalized recommendation of social software items based on social relations," in *Proceedings of the third ACM conference on Recommender systems*, 2009, pp. 53–60.
- [25] N. Siegmund, N. Ruckel, and J. Siegmund, "Dimensions of software configuration: on the configuration context in modern software development," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2020, pp. 338–349.
- [26] L. Lamontagne and I. Abi-Zeid, "Combining multiple similarity metrics using a multicriteria approach," in *European Conference on Case-Based Reasoning*. Springer, 2006, pp. 415–428.
- [27] M. Fu, G. Luo, X. Zheng, T. Zhang, D. Yu, and M. Kim, "An ensemble learning approach for software semantic clone detection," *arXiv preprint arXiv:2010.04336*, 2020.
- [28] D. Bär, C. Biemann, I. Gurevych, and T. Zesch, "Ukp: Computing semantic textual similarity by combining multiple content similarity measures," in * *SEM 2012: The First Joint Conference on Lexical and Computational Semantics—Volume 1: Proceedings of the main conference and the shared task, and Volume 2: Proceedings of the Sixth International Workshop on Semantic Evaluation (SemEval 2012)*, 2012.
- [29] R. Santos, P. Murrieta-Flores, and B. Martins, "Learning to combine multiple string similarity metrics for effective toponym matching," *International journal of digital earth*, vol. 11, no. 9, pp. 913–938, 2018.
- [30] J. Martinez-Gil, "Advanced detection of source code clones via an ensemble of unsupervised similarity measures," *arXiv preprint arXiv:2405.02095*, 2024.
- [31] D. Altarawy, H. Shahin, A. Mohammed, and N. Meng, "Lascad: Language-agnostic software categorization and similar application detection," *Journal of Systems and Software*, vol. 142, pp. 21–34, 2018.
- [32] Y. Ma, S. Fakhoury, M. Christensen, V. Arnaudova, W. Zogaan, and M. Mirakhorli, "Automatic classification of software artifacts in open-source applications," in *Proceedings of the 15th International Conference on Mining Software Repositories*, 2018, pp. 414–425.
- [33] P. T. Nguyen, J. Di Rocco, R. Rubel, and D. Di Ruscio, "Crosssim: Exploiting mutual relationships to detect similar oss projects," in *2018 44th Euromicro conference on software engineering and advanced applications (SEAA)*. IEEE, 2018, pp. 388–395.
- [34] "Esm - ensemble similarity models," 2025, accessed: March 11, 2025. [Online]. Available: https://anonymous.4open.science/r/sim_cal-8D30
- [35] "Maven model - introduction," 2025, accessed: June 1, 2025. [Online]. Available: <https://maven.apache.org/ref/current/maven-model/>
- [36] Travis-ci. Travis ci build config reference. [Online]. Available: <https://config.travis-ci.com/>
- [37] "Github - snakeyaml/snakeyaml," 2025, accessed: June 1, 2025. [Online]. Available: <https://bitbucket.org/asomov/snakeyaml>
- [38] Github-actions. Github actions documentation. [Online]. Available: <https://docs.github.com/en/actions>
- [39] R. A. Davis, K.-S. Lii, and D. N. Politis, "Remarks on some nonparametric estimates of a density function," in *Selected Works of Murray Rosenblatt*. Springer, 2011, pp. 95–100.
- [40] X. Wan, W. Wang, J. Liu, and T. Tong, "Estimating the sample mean and standard deviation from the sample size, median, range and/or interquartile range," *BMC medical research methodology*, 2014.
- [41] S. Patro and K. K. Sahu, "Normalization: A preprocessing stage," *arXiv preprint arXiv:1503.06462*, 2015.
- [42] J. Hauke and T. Kossowski, "Comparison of values of pearson's and spearman's correlation coefficients on the same sets of data," *Quaestiones geographicae*, vol. 30, no. 2, pp. 87–93, 2011.
- [43] A. Böttcher and D. Wenzel, "The frobenius norm and the commutator," *Linear algebra and its applications*, vol. 429, no. 8-9, 2008.
- [44] F. Mémoi, "Gromov-hausdorff distances in euclidean spaces," in *2008 IEEE Computer Society Conference on Computer Vision and Pattern Recognition Workshops*. IEEE, 2008, pp. 1–8.
- [45] J. C. Gower, "Properties of euclidean and non-euclidean distance matrices," *Linear algebra and its applications*, vol. 67, pp. 81–97, 1985.
- [46] S. Xia, Z. Xiong, Y. Luo, G. Zhang et al., "Effectiveness of the euclidean distance in high dimensional spaces," *Optik*, vol. 126, no. 24, 2015.
- [47] A. Kumar, Y. S. Ingle, A. Pande, and P. Dhule, "Canopy clustering: a review on pre-clustering approach to k-means clustering," *Int. J. Innov. Adv. Comput. Sci.(IJIACS)*, vol. 3, no. 5, 2014.
- [48] G. Zhang, C. Zhang, and H. Zhang, "Improved k-means algorithm based on density canopy," *Knowledge-based systems*, vol. 145, 2018.
- [49] K. R. Shahapure and C. Nicholas, "Cluster quality analysis using silhouette score," in *2020 IEEE 7th international conference on data science and advanced analytics (DSAA)*. IEEE, 2020, pp. 747–748.